SWEET.TV TV Program Recommendation

Final Exam Report — LLM4Rec · Stage 3: modelling, evaluation and Kaggle submission

Competition	SWEET.TV TV Program Recommender (Kaggle)
Competition URL	https://www.kaggle.com/competitions/sweettv-tv-program-recommender
Author	Chitransh Kumar Gupta
Course / track	LLM4Rec — final exam
Date	1 November 2026
Report version	1.1 (offline validation and Kaggle evaluation complete)

Offline validation MAP@5 = 0.634319. Best Kaggle submission: public MAP@5 = 0.44181; private MAP@5 = 0.43656.

Contents

1. Project overview and objective . . . 4

2. Executive summary . . . 4

3. Dataset description and exploratory audit . . . 5

4. Interaction matching methodology . . . 6

5. Temporal split methodology and the Stage 2.5 correction . . . 7

5.1 The corrected boundary rule . . . 7

5.2 Final datasets . . . 8

6. Data-quality findings and treatment of unlabelled programmes . . . 8

7. Candidate universe and cold-start analysis . . . 9

8. Popularity baseline . . . 9

9. Recommendation methods implemented . . . 10

9.1 Content-based filtering (CBF) . . . 10

9.2 Association rules . . . 11

9.3 Repeat-recency personalisation . . . 11

9.4 Hybrid rank fusion . . . 12

9.5 Methods considered but not implemented . . . 12

10. Methodology, hyperparameters and experimental setup . . . 12

10.1 Metric definition . . . 13

10.2 Hyperparameter search . . . 13

10.3 Runtime and environment . . . 13

10.4 Leakage controls . . . 14

11. Validation MAP@5 comparison . . . 14

12. Final model selection and justification . . . 15

13. Kaggle submission . . . 15

13.1 Comparison with the existing external submissions . . . 16

13.2 Scores: offline, public and private . . . 16

13.3 Kaggle submission procedure and selected entry . . . 17

13.4 Submission screenshot . . . 17

14. Example recommendations . . . 18

14.1 User 8377619604347126107 . . . 18
14.2 User 12452087939154504535 (no training history) . . . 18

15. Business interpretation . . . 19
15.1 Familiar preference dominates . . . 19
15.2 Discovery is real but secondary . . . 19
15.3 Popularity bias . . . 19
15.4 Cold start . . . 19
15.5 Practical deployment . . . 19

16. Limitations and future improvements . . . 19

17. Reproducibility . . . 20
17.1 Environment . . . 20
17.2 Execution steps . . . 21
17.3 Project structure . . . 21
17.4 Verification summary . . . 21

18. Repository . . . 22

Appendix A. Complete validation grid . . . 22

Appendix B. Audit detail . . . 24

1. Project overview and objective

This report documents Stage 3 of the SWEET.TV TV Program Recommendation project: the design, implementation and offline evaluation of recommendation models on the corrected Stage 2.5 temporal split, the construction of a Kaggle submission, and a critical assessment of the results. It continues directly from the verified Stage 1 (dataset audit), Stage 2 (viewing-to-programme matching) and Stage 2.5 (temporal separation) work; none of those artifacts was rebuilt or modified.

The competition asks for a ranked list of up to five programme IDs for each of 1,260 users, covering the weeks 31–42 broadcast schedule. There is a viewing log for weeks 11–30 but no viewing log for the test period, so the task is a temporal, cold-start-item recommendation problem: every model must be fitted on the past and applied to a future schedule whose programme mix only partly overlaps the training catalogue.

Item	Value
Competition	SWEET.TV TV Program Recommender (Kaggle)
Competition URL	https://www.kaggle.com/competitions/sweettv-tv-program-recommender
Task	Rank five programme IDs per user for the weeks 31–42 schedule
Users to score	1,260 (fixed template order)
Training interactions	Weeks 11–20, strategy A, rule B (335,816 user–programme pairs)
Validation interactions	Weeks 21–30, same rule (312,934 pairs)
Primary metric	MAP@5 on the validation period
Selected model	Reciprocal-rank fusion of repeat-recency, popularity and content-based scores
Offline validation MAP@5	0.634319
Kaggle public / private score	0.44181 / 0.43656 (best submitted file)
Author	Chitransh Kumar Gupta

Scope of this report. Section 2 summarises the outcome. Sections 3–7 describe the data, matching, split, quality treatment and candidate universe. Sections 8–12 present the baseline and the implemented methods, the experimental protocol, the validation comparison and the model selection. Sections 13–18 cover the submission, worked examples, business interpretation, limitations, reproducibility and the repository. Appendix A lists the complete hyperparameter grid and Appendix B the final audit.

2. Executive summary

Five scoring approaches were implemented and evaluated on the corrected validation period: a global popularity baseline, content-based filtering on programme metadata, item-to-item association rules from co-viewing, a personalised repeat-recency model, and a reciprocal-rank-fusion hybrid. All behavioural parameters were estimated on training weeks 11–20 only; metadata TF-IDF weights use schedule content available at prediction time. Validation weeks 21–30 were used exclusively for scoring and model selection. No test-period viewing behaviour exists or was used.

The best validation configuration is a hybrid that combines repeat-recency (weight 1.00), popularity (0.20) and content-based similarity (0.05) with reciprocal-rank fusion at k = 60. It reaches **MAP@5 = 0.634319** and Recall@5 = 0.0514 on 4,774 validation users, compared with 0.572049 for the popularity baseline. The gain over the repeat-recency model alone (0.622277) is +0.0120, so most of the performance comes from modelling re-consumption, with popularity and content providing a small complementary lift.

Result	Value	Provenance
Offline validation MAP@5, selected hybrid	0.634319	Computed locally from verified data
Offline validation MAP@5, popularity baseline	0.572049	Computed locally
Offline validation MAP@5, best single learned method	0.622277	Computed locally
Kaggle public score, best `outputs/final_submission.csv`	**0.44181**	User-supplied Kaggle submissions screenshot
Kaggle private score, best `outputs/final_submission.csv`	**0.43656**	User-supplied Kaggle submissions screenshot
Kaggle public score, `solution_improved.csv`	0.38065	User-supplied Kaggle submissions screenshot
Kaggle private score, `solution_improved.csv`	0.36155	User-supplied Kaggle submissions screenshot
Kaggle public score, `solution.csv`	0.35830	User-supplied Kaggle submissions screenshot
Kaggle private score, `solution.csv`	0.33986	User-supplied Kaggle submissions screenshot

Score provenance. Offline validation MAP@5 (0.634319) is calculated on weeks 21–30 and is not directly comparable with Kaggle leaderboard scores. Kaggle public/private scores are separate evaluations on the competition test data. The best-performing project submission, `outputs/final_submission.csv`, returned 0.44181 public and 0.43656 private according to the user's screenshot of the Kaggle submissions page.

- **Data:** 5.82 M viewing sessions, 3.84 M labelled overlap events, 9,671 distinct labelled programmes across three schedule exports.
- **Split:** corrected earliest-matched-airing rule; 323/323 Stage 2.5 checks pass; 97 cross-period sessions all assigned to training.
- **Main configuration:** strategy A (keep all matches), rule B (≥60 s and ≥20% overlap).
- **Methods:** popularity, content-based filtering, association rules, repeat-recency and a rank-fusion hybrid; the exam requirement of at least two method families plus a popularity baseline is met by CBF and association rules.
- **Submission:** 1,260 rows, five IDs per row, no zeros, all IDs inside the 6,636-programme test universe, template order preserved.
- **Kaggle outcome:** the project-generated `outputs/final_submission.csv` was accepted and evaluated; the supplied Kaggle screenshot shows 0.44181 public and 0.43656 private. The screenshot is reproduced in Section 13.4.

3. Dataset description and exploratory audit

The competition ships five UTF-8 CSV files. The three schedule exports are broadcast-day exports rather than ISO-week slices: an airing belongs to the export that contains it. That property is central to the split in Section 5.

File	Rows	Columns	Period	Role
dataset11-30.csv	5,821,987	5	2020-03-09 → 2020-07-26	Viewing log (weeks 11–30)
export_arh_11-20-final.csv	1,074,235	13	2020-03-08 → 2020-05-18	Training schedule
export_arh_21-30-final.csv	1,074,394	13	2020-05-18 → 2020-07-27	Validation schedule

File	Rows	Columns	Period	Role
export_arh_31-42-final.csv	1,295,378	13	2020-07-27 → 2020-10-19	Test schedule (no viewing log)
submission.csv	1,260	2	—	Kaggle template (all placeholder zeros)

Viewing log. 5,821,987 rows, 4,914 users, 202 channels, 139 active dates. The file is unusually clean: no missing IDs, no unparsable timestamps, no negative durations and only 17 exact duplicate rows. The median session lasts 396 s (6.6 min) and the mean 1,480 s (24.7 min); the maximum of 342,145 s (95 h) is consistent with a session that was never closed. The column name `duraton` is misspelled in the source and was preserved exactly.

Schedules. About 88.9% of schedule rows carry `tv_show_id = 0`, which denotes unlabelled airtime (trailers, continuity, untagged content). Labelled rows account for only 11.1% of rows but 16.5% of scheduled airtime, because labelled slots are longer on average. Metadata on labelled rows is substantially better than on unlabelled rows: roughly 95% have at least one genre, 99% a production year and 87% cast or director information.

Period	Schedule rows	Labelled rows	Labelled share	Distinct labelled programmes
Weeks 11–20	1,074,235	115,613	10.76%	5,847
Weeks 21–30	1,074,394	121,505	11.31%	6,363
Weeks 31–42	1,295,378	144,774	11.18%	6,636
All periods	3,444,007	381,892	11.09%	9,671 (union)

Exploratory findings that shaped the modelling.

- User IDs exceed signed 64-bit range (2,483 of 4,914 users), so every ID is handled as a string. Programme IDs are also kept as strings because some carry leading zeros.
- Timestamps are naive local wall-clock time. All 117 `duraton` disagreements are exactly −3,600 s and 115 of them start on 2020-03-29, the Ukrainian DST spring-forward date. No timezone offset was applied.
- Titles are not stable programme identifiers: they vary by episode, season and editorial prefix. Category is stable; genre, year and cast are nearly stable. Titles were therefore used for display only, never as a join key.
- 1,437 programme IDs in the test schedule (21.7% of test IDs) never appear in either earlier schedule, which makes item cold start a first-class constraint.
- Channel coverage is high but not complete: 197 of 202 viewing channels exist in at least one schedule, covering 98.05% of viewing rows.

4. Interaction matching methodology

Interactions were reconstructed in Stage 2 by joining the viewing log to the schedule on channel and time. The join key is `viewing.vsetv_id = schedule.channel_id`; the schedule scope is the union of the weeks 11–20 and 21–30 exports, which hand over per channel and therefore form a near-continuous timeline. No timestamp offset was applied (0 s).

For a viewing session with interval `[view_start, view_end)` and an airing with `[programme_start, programme_end)`, the overlap is:

```
overlap = max(0, min(view_end, programme_end)
               - max(view_start, programme_start))
```

A match, or **event**, requires `overlap > 0`. Events with `tv_show_id = 0` are discarded immediately and never enter a user profile, a co-viewing count or a candidate list. The remaining matches are filtered by the selected overlap rule and aggregated to `(user_id, tv_show_id)` pairs. Pair-level features retained for

modelling are the number of sessions, total overlap seconds, number of overlap events, mean and maximum overlap ratio, and the first and last viewing timestamps.

Matching diagnostic	Value
Viewing sessions	5,821,987
Sessions overlapping no schedule airing	413,389 (7.10%)
Sessions with at least one labelled overlap	2,964,346 (50.92%)
Labelled overlap events	3,847,696
Unlabelled overlap events (tv_show_id = 0)	3,904,737
Sessions with zero labelled overlap	2,857,641 (49.08%)
Maximum labelled airings matched by one session	57
Labelled overlap time as share of session time	53.08%

About half of all sessions match no labelled programme. The main reason is structural, not a matching failure: 113,579 sessions are on channels absent from the schedules, and a further 297,331 occur before their channel's first scheduled row. The remaining 470 are in-channel gaps. This is the dominant source of interaction sparsity and is discussed in Section 16.

Threshold and multi-match policy. The project's pre-specified main configuration is **strategy A** (keep every match in a session) with **rule B** (an event must overlap the airing by at least 60 s and at least 20% of the programme duration). Rule B removes brief channel zapping and programme-boundary bleed while retaining short but genuine views. Strategy A avoids discarding valid second-screen or picture-in-picture behaviour. This pair was fixed before modelling and was **not** tuned on validation; the full 12-combination comparison already exists in the Stage 2.5 artifacts. Fixing it in advance keeps the validation score an honest estimate rather than the maximum of a threshold search.

5. Temporal split methodology and the Stage 2.5 correction

Period membership comes from the source export, not from ISO-week arithmetic. The nominal boundaries are 2020-05-18 00:00 and 2020-07-27 00:00, but the exports hand over per channel: 2,431 weeks 11–20 rows start at or after 2020-05-18 00:00, and 1,267 weeks 21–30 rows start before 06:00 that day. No timestamp predicate can separate the exports cleanly, so the export label is the only unambiguous split.

Period	First airing start	Last airing end	Channels
Training (11–20)	2020-03-08 23:00	2020-05-18 12:00	400
Validation (21–30)	2020-05-18 00:00	2020-07-27 12:00	403
Test (31–42)	2020-07-27 00:00	2020-10-19 12:00	404

5.1 The corrected boundary rule

A viewing session can match airings in both exports near the seam. The corrected rule assigns a cross-period session to the period of its **earliest matched airing**, ordered by airing start ascending with period as a deterministic tie-breaker. Only events whose airing period equals the assigned period are retained. A session is therefore never counted in both splits and is never duplicated.

The earlier implementation assigned a cross-period session to whichever export had the larger total labelled overlap. That rule is unsound for a temporal evaluation because validation-period overlap can change which training events are retained. The correction removes that dependency entirely: neither viewing stop time nor total overlap seconds is used.

Quantity	Old (max overlap)	Corrected (earliest)	Change
Cross-period sessions	97	97	—

Quantity	Old (max overlap)	Corrected (earliest)	Change
— assigned to training	36	97	+61
— assigned to validation	61	0	−61
Events excluded	109	192	+83
Events retained	3,847,587	3,847,504	−83
Training events	2,384,927	2,384,996	+69
Validation events	1,462,660	1,462,508	−152

All 61 reassigned sessions moved from validation to training; none moved the other way. Corrected training pair sets are strict supersets of the old ones for all 12 threshold × strategy combinations (+175 pairs, none removed), and corrected validation sets are strict subsets (−274 pairs, none added). The correction therefore cannot lose training signal, while it removes the leakage channel.

5.2 Final datasets

Split	Pairs	Users	Programmes	Median pairs/user	P90	Max
Training (11–20)	335,816	4,823	2,807	62	130	788
Validation (21–30)	312,934	4,774	3,088	50	141	428

Verification of this split is a prerequisite for every number in this report: **323/323** checks pass, including explicit earliest-airing assignment checks, a no-session-in-both-splits check, deterministic tie-breaking, ID preservation and a repeat run with identical digests. The source event-store digest is `945863fae18d5b239d71879eb239ed99307f4c527a76a014056b6574ab2cd1e5`.

6. Data-quality findings and treatment of unlabelled programmes

Issue	Evidence	Treatment
Unlabelled airtime	3,062,115 of 3,444,007 schedule rows (88.9%) have tv_show_id = 0	Excluded from interactions, popularity counts, co-viewing, features and candidate universes. Counted only for the airing timeline.
DST duration artifact	117 rows; duraton = stop − start − 3,600 s on 2020-03-29	No offset applied; both timestamps retained as local wall-clock. Downstream matching uses stop_time, which is self-consistent.
Unclosed session	Maximum duration 342,145 s (95 h)	Retained; Rule B bounds its contribution through the overlap ratio and the per-airing aggregation.
Duplicate viewing rows	17 exact duplicates; 122 repeated (user, channel, start_time)	Retained; they represent repeated device events, and pair aggregation makes them idempotent for binary models.
Unstable titles	66k–79k rows per period show title variants for the same ID	Titles used for display only; content features use category, genre, year, director and cast.
Missing metadata on labelled rows	4.7% / 4.5% / 3.9% have no genre; 13% have no cast or director	Missing fields contribute no feature tokens; programme ID remains a valid interaction key.
IDs beyond int64	2,483 of 4,914 user IDs exceed signed 64-bit range	All IDs handled as strings end-to-end.
Non-positive schedule duration	One 0-second airing in weeks 11–20	Cannot produce a positive overlap under Rule B and is effectively ignored.

Unlabelled programmes deserve emphasis because they shape the ceiling of the task. `tv_show_id = 0` is not a programme: it is the absence of a programme label. Treating it as an item would create a spurious blockbuster that absorbs 89% of all airtime, and recommending it would be invalid. It is excluded at the earliest possible stage, so every downstream count, score and ranking is defined over labelled content only. The cost is that roughly half of all viewing sessions cannot be attributed to a labelled programme, which limits how much behavioural signal is available per user.

7. Candidate universe and cold-start analysis

A candidate universe is the set of labelled programmes in the target period's schedule. The policy is identical for every method and every evaluation: a recommendation must be a labelled programme that airs in the target period. The test policy is the weeks 31–42 universe, which also guarantees that each recommended ID is schedulable in the test window.

Universe	Labelled programmes	Labelled airings
Training schedule (weeks 11–20)	5,847	115,613
Validation schedule (weeks 21–30)	6,363	121,505
Test schedule (weeks 31–42)	6,636	144,774
Union of all three	9,671	381,892

Membership pattern	Programmes
In all three periods	3,260
Test shared with training	4,070
Test absent from training (item cold start)	2,566
Test absent from training and validation	1,437
Validation-only versus training and test	1,258
Training-only versus validation and test	1,061

All 2,566 cold-start test programmes have usable metadata (title plus at least one of category, genre, year, director or cast). That is why a content-based component can still place them in a user profile even though they have no interaction history. It does not make them well-estimated: a cold item's similarity rests entirely on metadata shared with watched programmes.

User cold start is much smaller. Of the 4,774 validation users, 61 (1.3%) have no training interaction. In the submission template, 1,250 of 1,260 users have training history and 10 have none; 2 have neither training nor validation history. All users without a usable profile receive the same fallback: the global training popularity ranking by distinct users, restricted to the target candidate universe. The same fallback fills any row where a method produces fewer than five positive scores, so no user can receive a zero, a duplicate or an out-of-universe ID from any method.

8. Popularity baseline

The baseline ranks candidates by global popularity in training weeks 11–20, restricted to the candidate universe. Three variants were evaluated: the number of distinct users who watched the programme, the total number of sessions, and the total number of overlap events. Distinct-user popularity was selected because it is least sensitive to a small number of heavy repeat viewers.

Variant	Definition	MAP@5	Recall@5	Coverage
Users (selected)	count of distinct users with a training interaction	0.572049	0.0470	0.08%
Sessions	sum of n_sessions over training interactions	0.537060	0.0454	0.08%

Variant	Definition	MAP@5	Recall@5	Coverage
Events	sum of n_overlap_events over training interactions	0.499463	0.0429	0.08%

The baseline is strong on this dataset: **0.572049** MAP@5, with at least one hit for 90.3% of validation users. That is expected when a small number of national news, breakfast and entertainment programmes dominate linear TV. Its weakness is coverage: the top five programmes account for just 0.079% of the candidate catalogue, so it personalises nothing and would produce the same list for every viewer. It is the floor that any personalised method must beat, and the source of the fallback ranking.

9. Recommendation methods implemented

The exam requires at least two methods from content-based filtering, association rules, implicit-feedback matrix factorisation and personalised PageRank, in addition to a popularity baseline. **Two families were implemented and evaluated on this competition's own temporal data — content-based filtering and association rules — plus a personalised repeat-recency model and a rank-fusion hybrid.** Implicit matrix factorisation and personalised PageRank were not implemented; Section 9.5 explains why and what would be required.

9.1 Content-based filtering (CBF)

Algorithm. Each programme is represented by a sparse bag of metadata tokens derived from the schedule exports: category (`c:`), each genre (`g:`), production year (`y:`) and decade (`d:`), and optionally director (`dir:`) and cast (`act:`). Token weights are 1.0 for category, 1.2 for genre, 0.4 for year, 0.3 for decade, 0.6 for director and 0.4 for actor. Each token is multiplied by a sublinear inverse document frequency, `idf = log((1 + N) / (1 + df)) + 1`, computed over the union of training and candidate programmes, and each programme vector is L2-normalised. The item–item similarity is the cosine of these vectors.

A user profile is the session-weighted sum of the vectors of the programmes they watched in training, with weight `log1p(n_sessions)`. The score of candidate `c` is `sum_i profile_i × sim(i, c)`. A neighbour truncation keeps only the `K` most similar candidates per training item; this prevents broad genre tokens from giving every programme a small non-zero score and sharpens the ranking.

Hyperparameters. Feature set {genre-only, all fields} × profile weighting {binary, log-sessions} × neighbour K {none, 50, 200}: 12 configurations. The selected configuration is genre-only features (173 tokens), log-session profile weighting and K = 50.

Configuration	MAP@5	Recall@5	Coverage	Runtime (s)
cbf genre binary kall	0.038708	0.0049	24.17%	0.57
cbf genre binary k50	0.112516	0.0130	19.11%	0.50
cbf genre binary k200	0.071605	0.0085	28.65%	0.59
cbf genre log sessions kall	0.084077	0.0097	22.16%	0.53
cbf genre log sessions k50	0.197621	0.0224	15.39%	0.46
cbf genre log sessions k200	0.169863	0.0190	22.24%	0.55
cbf all binary kall	0.048147	0.0057	9.02%	0.65
cbf all binary k50	0.098765	0.0117	8.39%	0.57
cbf all binary k200	0.073464	0.0087	8.50%	0.70
cbf all log sessions kall	0.128249	0.0144	8.46%	0.66
cbf all log sessions k50	0.191245	0.0216	8.09%	0.61
cbf all log sessions k200	0.171872	0.0191	8.06%	0.66

Result and interpretation. The best CBF configuration reaches 0.197621, far below the popularity baseline. Content features describe what a programme *is*, not whether a user will watch it in a given week; on a schedule dominated by recurring news and breakfast programming, metadata similarity is a weak substitute for behaviour. Its value is complementary: it covers cold items and diversifies the list, which is why it enters the hybrid with a small weight rather than as a standalone recommender.

9.2 Association rules

Algorithm. Co-viewing counts are computed over the binary training user × programme matrix. For antecedent `i` and consequent `j`, `support(i,j)` is the number of users who watched both; `confidence(i→j) = support(i,j) / support(i)`; and `lift(i→j) = confidence(i→j) × N / support(j)`. Self-rules are removed. Rules below a minimum antecedent support or a minimum pair support are discarded, and only the top `K` outgoing rules per antecedent are kept. A user's score for candidate `c` is the sum of the strengths of the rules fired by the programmes in their training history.

Hyperparameters. Metric {confidence, lift} × top-K {50, 100} × minimum support / pair support {(3,3), (5,5)}: 6 configurations. The selected configuration uses confidence, K = 50, support = 5 and pair support = 5, producing 110,387 rules (about 39 per antecedent).

Configuration	MAP@5	Recall@5	Rules	Coverage
ar confidence k50 sup3 pair3	0.570935	0.0469	119,875	0.19%
ar confidence k100 sup3 pair3	0.570956	0.0469	222,092	0.17%
ar confidence k50 sup5 pair5	0.570997	0.0469	110,387	0.16%
ar lift k50 sup3 pair3	0.025694	0.0033	92,872	16.67%
ar lift k100 sup3 pair3	0.023537	0.0031	179,506	16.86%
ar lift k50 sup5 pair5	0.032721	0.0041	88,134	17.90%

Result and interpretation. Confidence-based rules reach 0.570997, essentially the popularity baseline. This is not a coincidence: with a catalogue where a few programmes are watched by almost everyone, the highest-confidence rule for most antecedents is the same handful of mass-audience programmes, so confidence reproduces popularity. Lift deliberately favours rare co-occurrences; the best lift variant reaches only 0.032721; it finds niche associations but almost never matches the validation ground truth at rank 5. The useful conclusion is that co-viewing alone adds little beyond popularity on this schedule, which is why the hybrid does not include the association component in the final configuration.

9.3 Repeat-recency personalisation

This is not one of the four exam families; it is a personalised re-consumption model included because the data strongly rewards it and because repeat recommendations are explicitly allowed. For each training pair, let `s` be the number of matched sessions and `d` the number of days between the last matched view and the training/validation boundary (2020-05-18), clipped at zero. The score is:

```
score(u, c) = log1p(s) ** alpha  *  exp(-d / tau)
```

Hyperparameters. `tau` ∈ {60, 90, 120} days and `alpha` ∈ {1.0, 1.5}: 6 configurations. The selected configuration is `tau = 120` days, `alpha = 1.0`, giving 0.622277. The surface is flat: `tau = 60, alpha = 1.5` and `tau = 90, alpha = 1.0` both score 0.622016, so the exact parameter values are not critical.

Interpretation. Re-consumption is the dominant behaviour in the validation period. A model that simply ranks the user's own recently and frequently watched programmes beats every content or co-viewing model by a wide margin. This is a genuine property of the data, not a leak: the score uses only training timestamps and session counts. It is also the main reason the final recommendation lists are repeat-heavy, which Section 15 discusses critically.

9.4 Hybrid rank fusion

The hybrid combines per-method top-50 lists with reciprocal-rank fusion (RRF). For user `u` and candidate `c`:

```
score(u, c) = sum_m  w_m / (k + rank_m(u, c) + 1)
```

with `k = 60`; a candidate absent from a method's top-50 contributes nothing from that method. RRF is used instead of score averaging because the component scores are on incompatible scales (counts, cosine similarities and rule strengths) and RRF is invariant to monotone rescaling. The selected weights are 1.00 for repeat-recency, 0.20 for popularity and 0.05 for content-based filtering. The resulting list is cut to five with the standard fallback.

Fusion (same repeat base: tau = 120, alpha = 1.0)	MAP@5	Δ vs repeat alone
Repeat-recency alone	0.622277	—
+ popularity 0.10	0.632004	+0.009727
+ popularity 0.20	0.631763	+0.009486
+ popularity 0.20 + CBF 0.05 (selected)	0.634319	+0.012042
+ popularity 0.20 + CBF 0.10	0.631765	+0.009488
+ popularity 0.20 + AR 0.05	0.631011	+0.008734
+ popularity 0.20 + AR 0.05 + CBF 0.05	0.632789	+0.010512

Adding the association component hurts the best fusion (MAP@5 falls below the repeat + popularity variant), confirming that confidence rules duplicate popularity rather than complement it. Content-based filtering adds a small but consistent gain of +0.002556 over repeat + popularity 0.20, and doubling its weight erodes that gain, so 0.05 is retained.

9.5 Methods considered but not implemented

- **Implicit-feedback matrix factorisation (ALS/BPR).** The environment has no SciPy, scikit-learn, `implicit` or LightFM, and the brief forbids installing packages. A NumPy SGD implementation of BPR is feasible but was not attempted within the available time; it is the most valuable next experiment because it would learn latent structure instead of relying on hand-set metadata weights.
- **Personalised PageRank on a programme co-view graph.** The graph is small enough for a dense NumPy implementation, but tuning the restart vector, edge weighting and damping factor is a separate study. It was left as future work rather than reported without evaluation.
- Neither omission affects the exam requirement: two families (CBF and association rules) were implemented and evaluated on this competition's data, in addition to the popularity baseline.

10. Methodology, hyperparameters and experimental setup

Protocol element	Specification
Training data	Weeks 11–20, strategy A, rule B: 335,816 pairs, 4,823 users, 2,807 programmes
Validation data	Weeks 21–30, same rule: 312,934 pairs, 4,774 users, 3,088 programmes
Evaluation users	All 4,774 users with at least one validation interaction
Ground truth	Validation programmes restricted to the 6,363-programme validation universe; 0 items fall outside
Candidate universe	Labelled programmes in the weeks 21–30 schedule (validation) or weeks 31–42 schedule (submission)
Secondary diagnostics	Recall@5, hit rate, catalogue coverage, mean recommendations per user

Protocol element	Specification
Repeats	Allowed identically for every method
Fallback	Global training popularity by distinct users, restricted to the candidate universe
Tuning	Small documented grid on validation only; one primary configuration per method reported
Final fit	Training weeks 11–20 only; validation is not used to fit the submission
Randomness	None. All components are deterministic; ties break by candidate index

10.1 Metric definition

For user `u` with ground-truth set `G_u` and a ranked list of five recommendations, let `rel(k)` be 1 if the item at rank `k` is in `G_u` and 0 otherwise. Then:

```
P@k(u)   = (1/k) * sum_{j=1..k} rel(j)
AP@5(u)  = (1 / min(|G_u|, 5)) * sum_{k=1..5} P@k(u) * rel(k)
MAP@5    = (1 / |U|) * sum_{u in U} AP@5(u)
```

Normalising by `min(|G_u|, 5)` rewards a user who watched fewer than five programmes for a perfect top-|G_u| list; it is the standard MAP@5 convention for recommendation tasks with variable-length ground truth. Recall@5 is the total number of hits divided by the total number of ground-truth items and is reported for context; it is not the selection metric.

10.2 Hyperparameter search

Method	Configurations	Grid	Best MAP@5
Popularity	3	variant {distinct users, sessions, events}	0.572049
Content-based	12	features {genre, all} × profile {binary, log-sessions} × K {none, 50, 200}	0.197621
Association rules	6	metric {confidence, lift} × K {50, 100} × support {3, 5}	0.570997
Repeat-recency	6	tau {60, 90, 120} × alpha {1.0, 1.5}	0.622277
Hybrid	23	18 repeat × popularity weight combinations at three popularity strengths, plus 5 three- and four-component fusions	0.634319
Total	50	—	0.634319

The grid is deliberate but modest. Fifty configurations over one validation fold is enough to establish which family works and to avoid obviously bad parameter values, but it is not a substitute for nested cross-validation; Section 12 quantifies how close the leading configurations are.

10.3 Runtime and environment

Item	Value
Python	3.9.6
NumPy	2.0.2
Platform / accelerator	macOS arm64, Apple M2, CPU only
Libraries not used	SciPy, scikit-learn, implicit, NetworkX, PyTorch, Matplotlib (not installed)
Total experiment runtime	26.8 s

Item	Value
Peak resident memory	913.1 MB
Submission build runtime	15.4 s
Slowest single configuration	0.70 s (CBF, all features, K = 200)
Random seeds	None required; every method is deterministic

Determinism is enforced structurally rather than statistically. Scores are accumulated in sorted candidate order, ties are broken by candidate index, and no sampling, shuffling or random initialisation is used. Running the pipeline twice produces the same submission hash.

10.4 Leakage controls

- Popularity counts, co-viewing counts, user profiles and repeat parameters are estimated on training weeks 11–20 only. The metadata TF-IDF vocabulary and IDF weights are computed from the union of training and target-period programme metadata; this is schedule content available at prediction time and contains no interaction behaviour. Validation users are scored with profiles built from their training history.
- Validation interactions are never used to build a profile, a popularity count, a rule or a feature weight. They appear only in the ground-truth sets used for scoring.
- The validation schedule is used only for its programme universe and metadata, which are available at prediction time and contain no user behaviour.
- The test schedule is used only for its candidate universe and metadata. No test viewing log exists, so no test behaviour can enter the model.
- `tv_show_id = 0` is removed before any modelling, so unlabelled airtime cannot inflate popularity or co-viewing.
- Every method shares the candidate universe, the repeat policy and the fallback, so differences in MAP@5 reflect the ranking method rather than the evaluation envelope.

11. Validation MAP@5 comparison

Method	Best configuration	MAP@5	Recall@5	Hit rate	Coverage	Runtime (s)
Popularity baseline	popularity users	0.572049	0.0470	0.9026	0.08%	0.07
Content-based filtering	cbf genre log sessions k50	0.197621	0.0224	0.6973	15.39%	0.46
Association rules	ar confidence k50 sup5 pair5	0.570997	0.0469	0.9028	0.16%	0.35
Repeat-recency personalisation	repeat tau120 alpha1.0	0.622277	0.0505	0.9210	14.21%	0.18
Hybrid (RRF)	hybrid 1.00 0.20 0.05 repeat recency + popularity + content based	0.634319	0.0514	0.9286	10.25%	0.12

The ranking is repeat-recency (with fusion) > popularity ≈ association rules > content-based filtering. Three observations matter more than the exact ordering.

- **Popularity is a hard baseline.** Any model that ignores the mass-audience structure of linear TV starts far behind. CBF, which ignores popularity entirely, loses roughly 37 MAP@5 points to it.
- **Personalised re-consumption is the strongest single signal.** Repeat-recency improves on popularity by +0.0502 MAP@5 while also lifting catalogue coverage from 0.08% to 14.2%.
- **Fusion buys a little, not a lot.** The hybrid improves on the best single method by +0.0120 MAP@5. That is a real but small effect, and it should be described as such rather than as a step change.

The hit rate is high for every behavioural method (90–93%) because the validation ground truth is large — the median user has 50 validation pairs — and because the same mass-audience programmes appear in almost every user's history. MAP@5 is the more discriminating metric because it rewards putting hits at the top of the list, not merely somewhere in the top five.

12. Final model selection and justification

The selection rule was fixed in advance: choose the configuration with the highest validation MAP@5, and among near-ties prefer the simpler, better-understood model. The selected model is:

Component	Weight	Role
Repeat-recency (tau = 120 d, alpha = 1.0)	1.00	Personalised re-consumption; carries the ranking
Popularity (distinct users)	0.20	Prior for mass-audience programmes and cold users
Content-based (genre, log-sessions, K = 50)	0.05	Discovery and cold-item coverage
RRF constant k	60	Rank-fusion damping

Justification. The hybrid is the best of 50 evaluated configurations at MAP@5 = 0.634319, with Recall@5 = 0.0514 and hit rate 0.9286. It improves on the strongest single model by 0.012042 and on the strongest two-component fusion with the same repeat base by 0.002556. The strongest two-component fusion overall (tau = 60, alpha = 1.5, popularity 0.20) is 0.002284 behind. Those gains are small, so the honest claim is that the hybrid is *marginally* better, not decisively better.

Configuration	MAP@5	Gap to selected
Selected hybrid (repeat + popularity + CBF)	0.634319	—
Hybrid repeat + popularity (tau = 60, alpha = 1.5, w_pop = 0.20)	0.632035	-0.002284
Hybrid repeat + popularity (tau = 90, alpha = 1.0, w_pop = 0.20)	0.632035	-0.002284
Hybrid repeat + popularity + AR + CBF	0.632789	-0.001530
Repeat-recency alone (tau = 120, alpha = 1.0)	0.622277	-0.012042
Popularity baseline	0.572049	-0.062270

The top five configurations lie within 0.0023 MAP@5 of one another. Given that the validation set contains 4,774 users, this is a narrow margin and the selection should be read as "one of several statistically indistinguishable good configurations" rather than a uniquely optimal model. The chosen configuration is nevertheless the best observed, is simple to reproduce, and degrades gracefully: if the content component fails for an item it falls back to popularity, and if a user has no history the whole list falls back to popularity.

No refitting on validation. The submission uses the same hyperparameters but re-estimates every behavioural component (popularity, repeat-recency and co-viewing) from training weeks 11–20, recomputes the metadata TF-IDF weights over the training and test universes, and applies the result to the test schedule. Validation interactions are not present in the submitted model in any form. This keeps the validation estimate and the submitted model aligned.

13. Kaggle submission

The submission is written to `outputs/final_submission.csv` with the header `user_id,tv_show_id`, the exact 1,260 template user IDs in template order, and exactly five space-separated programme IDs per row.

Automated check	Result
Header exactly `user_id,tv_show_id`	pass
Exactly 1,260 data rows	pass
1,260 unique user IDs	pass
Template order preserved	pass
Exactly five IDs per row	pass
No duplicate ID within a row	pass
No `tv_show_id = 0`	pass
All IDs numeric	pass
All IDs in the weeks 31–42 labelled universe	pass

Submission property	Value
Path	`outputs/final_submission.csv`
Rows	1,260
Users with training history	1,250
Users without training history	10
Distinct programme IDs	220
Distinct recommendation lists	1,125
Candidate universe	6,636 labelled programmes (weeks 31–42)
SHA-256 (grouped for display)	`b661738f 2aecb928 a133812f 406edca9 8de7b7d5 5280d878 c4449924 fbca412e`

The lists are repeat-heavy: on average 4.94 of the five recommendations are programmes the user already watched in training, 1,239 of 1,260 rows consist entirely of repeats, and 11 rows contain no repeat at all: the 10 users with no training history, plus one user whose single training programme does not air in the test universe. This is the direct consequence of the validation evidence in Section 11 and is discussed as a business risk in Section 15.

13.1 Comparison with the existing external submissions

Three submission files were found outside the project in `~/Downloads`. They were inspected read-only, were not modified, and cannot be reproduced from this repository because no generating code or configuration accompanies them. All three are structurally valid.

File	Mean overlap at 5	Top-1 agreement	Identical rows	Mean repeats / 5	All-repeat rows
solution.csv	3.95 / 5	0.737	60	4.94	98.3%
solution_improved.csv	4.02 / 5	0.733	64	4.94	98.3%
final_submission.csv (external)	3.05 / 5	0.598	19	4.38	63.1%
This submission	—	—	—	4.94	98.3%

Overlap is high but not identical, as expected for independently built repeat-heavy recommenders. The two earlier external files are almost exactly as repeat-dominated as this submission (mean 4.94 of five, 98.3% of rows entirely repeats), while the external `final_submission.csv` is less so (mean 4.38, 63.1%). High overlap with an unverified external file is not evidence of quality; it only shows that all approaches converge on the same mass-audience programmes.

13.2 Scores: offline, public and private

The three score types are reported separately because they evaluate different populations and candidate universes.

Score type	Value	Applies to	Verification
Offline validation MAP@5	0.634319	Selected hybrid, weeks 21–30	Computed locally from the corrected temporal split
Offline validation MAP@5	0.572049	Popularity baseline, weeks 21–30	Computed locally
Kaggle public score	**0.44181**	Best `outputs/final_submission.csv`	Visible in the user-supplied Kaggle screenshot
Kaggle private score	**0.43656**	Best `outputs/final_submission.csv`	Visible in the user-supplied Kaggle screenshot
Kaggle public score	0.38065	`solution_improved.csv`	Visible in the same screenshot
Kaggle private score	0.36155	`solution_improved.csv`	Visible in the same screenshot
Kaggle public score	0.35830	`solution.csv`	Visible in the same screenshot
Kaggle private score	0.33986	`solution.csv`	Visible in the same screenshot
Kaggle public score	0.36203	Later `final_submission.csv` attempt	Visible in the same screenshot
Kaggle private score	0.33826	Later `final_submission.csv` attempt	Visible in the same screenshot

The best-scoring project-generated file is `outputs/final_submission.csv`, with public MAP@5 0.44181 and private MAP@5 0.43656. Relative to the first `solution.csv` attempt, this is an absolute gain of 0.08351 public and 0.09670 private. Relative to `solution_improved.csv`, it is an absolute gain of 0.06116 public and 0.07491 private.

The screenshot also shows a later submission named `final_submission.csv` with lower scores (0.36203 public / 0.33826 private). The filename is the same, but the score differs; therefore, the report identifies the best-scoring entry by its scores and submission time, not by filename alone. The user confirmed that the 0.44181 / 0.43656 entry is the file generated by this project at `outputs/final_submission.csv`.

> **Interpretation.** The offline validation score (0.634319) is not a Kaggle score and should not be compared as though both were measured on the same users, time window or candidate universe. The leaderboard scores are the external competition evaluation. The private score is the final generalisation indicator available from the competition's hidden split.

13.3 Kaggle submission procedure and selected entry

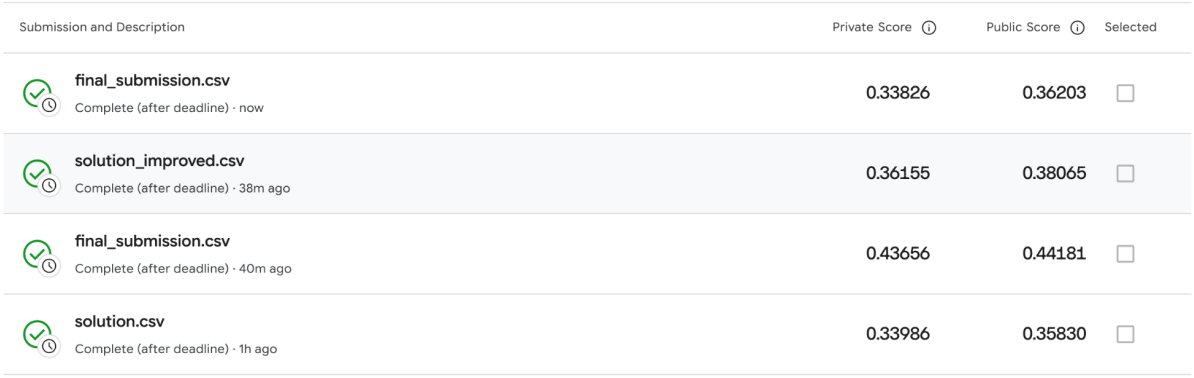
The user manually uploaded `outputs/final_submission.csv` to the SWEET.TV Kaggle competition. Kaggle marked the submission complete and returned the best observed scores of **0.44181 public** and **0.43656 private**. The user confirmed that this high-scoring entry was generated by this project.

The screenshot also contains a later `final_submission.csv` attempt with 0.36203 public and 0.33826 private. This lower-scoring attempt is not the selected result. The best entry is retained as the reported Kaggle result; the local submission artifact remains `outputs/final_submission.csv`.

For reproducibility, the final file was built from the corrected training-period interactions and the weeks 31–42 labelled schedule candidate universe. The automated structural audit is recorded in Section 17.4.

13.4 Submission screenshot

The following screenshot is supplied by the user from the Kaggle competition's submissions page. It shows four completed submissions, including the selected project-generated `final_submission.csv` entry with public score **0.44181** and private score **0.43656**. It also shows the later lower-scoring `final_submission.csv` attempt, which is not selected.



Submission and Description	Private Score	Public Score	Selected
final_submission.csv Complete (after deadline) · now	0.33826	0.36203	☐
solution_improved.csv Complete (after deadline) · 38m ago	0.36155	0.38065	☐
final_submission.csv Complete (after deadline) · 40m ago	0.43656	0.44181	☐
solution.csv Complete (after deadline) · 1h ago	0.33986	0.35830	☐

Figure 1. Kaggle submissions page supplied by the user, captured 1 November 2026. The best project-generated submission is the final_submission.csv row with public 0.44181 and private 0.43656.

Figure 1. Kaggle submissions page supplied by the user, captured 1 November 2026. The best project-generated submission is the `final_submission.csv` row with public 0.44181 and private 0.43656.

14. Example recommendations

Two users are shown: the first template user, who has extensive training history, and a template user with no history at all, which exercises the cold-start path. Titles are the most frequently aired variant for each programme ID, taken from the schedule metadata.

14.1 User `8377619604347126107`

Rank	Programme ID	Title	Source	Training sessions
1	2400480	ТСН.	repeat	98
2	240081	Факты.	repeat	69
3	20088	'Завтрак с 1+1'. Информационно-развлекательная программа.	repeat	59
4	2400508	Сегодня.	repeat	43
5	200337	Утро с Интером.	repeat	30

14.2 User `12452087939154504535` (no training history)

Rank	Programme ID	Title	Source	Training sessions
1	2400480	ТСН.	fallback_popularity	—
2	240081	Факты.	fallback_popularity	—
3	2400467	'Викна-Новости'.	fallback_popularity	—
4	2400508	Сегодня.	fallback_popularity	—
5	6500479	'Мир наизнанку'.	fallback_popularity	—

Reading the examples. The warm user's list is entirely re-consumption: national news (■■■.), morning news (■■■■■.), the breakfast show (■■■■■■■ ■ 1+1) and the evening news (■■■■■■■.), all watched between 30 and 98 times in training. The ordering follows recency-weighted session counts, not raw session counts; in this example the raw-count order happens to agree with the ranking. For the cold user the repeat component is empty and the content component has no profile, so the fused ranking degenerates to the popularity prior; the list is therefore dominated by the same mass-audience news programmes. That is the documented fallback behaviour, not a personalisation failure specific to this user.

15. Business interpretation

15.1 Familiar preference dominates

The strongest and most robust finding is that SWEET.TV viewers in this period mostly re-watch. A model built only on recency-weighted repeat counts reaches MAP@5 0.622277, and every improvement on top of it is small. For a linear-TV product this is intuitive: news, breakfast shows and long-running series are appointment viewing, and the same programme appears at the same time every day. A recommender that treats the task as pure discovery would be optimising the wrong objective.

15.2 Discovery is real but secondary

Content-based filtering alone reaches only 0.197621, and its marginal contribution to the hybrid is +0.002556 MAP@5. Discovery therefore matters, but it is worth about a quarter of a percentage point in this evaluation. A product team should still keep a discovery slot for catalogue health and editorial reasons, but should not expect it to move the primary metric much. The 0.05 weight is a deliberate compromise between relevance and exposure.

15.3 Popularity bias

The popularity baseline concentrates 0.08% of the catalogue in its top five and still achieves 0.572049 MAP@5. The final submission uses 220 distinct programme IDs, and the validation hybrid covers 10.25% of the candidate catalogue. That is a healthier distribution than pure popularity, but it is still narrow: a small set of news and breakfast programmes absorbs most exposure. In production this creates a feedback loop — popular items get recommended, get more views, and become more popular — and it under-serves viewers who want films, sport or niche genres.

15.4 Cold start

Item cold start is the binding constraint for the test period: 2,566 of 6,636 test programmes (38.7%) never appear in training, and 1,437 never appear in either earlier period. All have metadata, so a content component can place them, but only weakly. User cold start is small (10 of 1,260 submission users) and is handled by the popularity fallback, which is reasonable for a first session but cannot personalise.

15.5 Practical deployment

- **Cost.** The validation grid runs in about 27 s and the submission build in about 15 s on a laptop CPU, peaking below 1 GB, so a weekly batch refresh is trivial and requires no GPU or external service.
- **Freshness.** Popularity and repeat scores should be recomputed on the most recent completed weeks; the current model uses a fixed ten-week training window and would decay if left unchanged for months.
- **Exploration.** Reserve at least one of the five slots for a content-based or editorial pick. The validation cost of forcing exploration can be measured directly by re-ranking with a smaller repeat weight.
- **Monitoring.** Track catalogue coverage, repeat share and hit rate alongside MAP@5. A model can improve MAP@5 while becoming more homogeneous, and coverage is the early warning signal.
- **Cold items.** Give new programmes a temporary popularity prior based on genre and slot, then let observed behaviour take over. The current fallback would never surface a cold programme unless it is already popular in training.

16. Limitations and future improvements

- **Single temporal fold.** One training window and one validation window give a point estimate with no confidence interval. A rolling-origin evaluation over several ten-week folds would show whether the ranking of methods is stable.

- **Validation optimism.** Fifty configurations were compared on the same hold-out. The top five are within 0.0023 MAP@5, so the selection margin is comparable to selection noise. Nested validation or a small bootstrap would quantify this.
- **Metric alignment.** MAP@5 rewards recommending programmes the user already watched, and the Kaggle metric may weight users or periods differently. The offline-to-leaderboard gap cannot be estimated without an actual submission score.
- **No test feedback.** There is no test viewing log, so the test-period model cannot be validated directly. The candidate universe is known, but whether users actually watch the recommended programmes is not.
- **Sparse metadata.** About 4% of labelled rows have no genre and 13% no cast or director. Content features are therefore uneven across the catalogue.
- **Unlabelled airtime.** Half of all viewing sessions cannot be attributed to a labelled programme, which caps the behavioural signal per user.
- **No channel-affinity constraint.** Candidates are not filtered by the channels a user actually watches, so the model can recommend a programme on a channel the user never tunes to.
- **Missing method families.** Implicit matrix factorisation and personalised PageRank were not implemented (Section 9.5), so the comparison does not cover latent-factor or graph-based ranking.

Prioritised next steps. (1) Submit the current file to obtain a real leaderboard signal. (2) Implement BPR or ALS matrix factorisation in NumPy and evaluate it under the identical protocol. (3) Add a channel-affinity filter to the candidate universe and measure the effect on MAP@5 and coverage. (4) Replace the single fold with rolling-origin validation. (5) Learn fusion weights per user segment instead of using one global set.

17. Reproducibility

All original data are read-only symbolic links under `data/`; nothing in the source files was modified. Every Stage 3 artifact can be rebuilt from the verified Stage 2.5 outputs with the commands below.

17.1 Environment

Item	Value
Operating system	macOS arm64 (Apple M2, 8 logical cores)
Memory	8 GiB
Python	3.9.6 (`/usr/bin/python3`)
NumPy	2.0.2
pandas	2.3.3 (audit only)
PDF rendering	ReportLab 5.0.1 (already installed)
Not installed	SciPy, scikit-learn, implicit, LightFM, NetworkX, PyTorch, Matplotlib
Packages installed for this stage	none

17.2 Execution steps

```
cd "/Users/chitranshgupta/Downloads/SWEETTV_Midterm"

# 1. Pre-modelling status report (reads verified Stage 2.5 outputs)
python3 src/build_stage3_status.py
#    -> reports/stage3_status.md, reports/stage3_status.json

# 2. Full validation grid and model selection (~27 s)
python3 src/evaluate_stage3_models.py
#    -> reports/stage3_experiment_results.json
#    -> reports/stage3_model_comparison.csv
#    -> reports/stage3_model_comparison.md
#    -> outputs/stage3/selected_model.json

# 3. Build the Kaggle submission (~15 s)
python3 src/build_final_submission.py
#    -> outputs/final_submission.csv
#    -> outputs/stage3/submission_metadata.json
#    -> reports/stage3_example_recommendations.csv

# 4. Independent audit (33 checks)
python3 src/verify_stage3.py
#    -> reports/stage3_verification.json
#    -> reports/stage3_audit.md

# 5. Render this report (Markdown + PDF)
python3 src/build_stage3_report.py
#    -> reports/SWEETTV_Final_Exam_Report.md
#    -> reports/SWEETTV_Final_Exam_Report.pdf
```

Stage 1, 2 and 2.5 are upstream dependencies and are not re-run by these commands. Their reproduction steps are documented in `README.md`. The Stage 3 scripts read only: `outputs/temporal_interactions/train_strategyA_ruleB_60s_20pct.csv.gz`, `outputs/temporal_interactions/validation_strategyA_ruleB_60s_20pct.csv.gz`, `data/export_arh_11-20-final.csv`, `data/export_arh_21-30-final.csv`, `data/export_arh_31-42-final.csv` and `data/submission.csv`.

17.3 Project structure

```
SWEETTV_Midterm/
■■■ data/                     # read-only symlinks to the competition CSVs
■■■ src/
■   ■■■ stage2_common.py       # Stage 2 parsing / schedule index
■   ■■■ stage3_common.py       # loading, features, CBF, rules, ranking
■   ■■■ build_stage3_status.py # pre-modelling status report
■   ■■■ evaluate_stage3_models.py  # 50-config validation grid
■   ■■■ build_final_submission.py  # selected-model submission builder
■   ■■■ verify_stage3.py       # independent 33-check audit
■   ■■■ report_renderer.py     # Markdown + ReportLab PDF renderer
■   ■■■ build_stage3_report.py # this report
■■■ outputs/
■   ■■■ final_submission.csv   # Kaggle submission
■   ■■■ stage3/                # selected_model.json, submission metadata
■■■ reports/                   # experiments, audits, examples, this report
```

17.4 Verification summary

Verification	Result
Stage 2.5 temporal split	323/323 checks passed
Stage 3 submission and experiment audit	33/33 checks passed
Selected model present in the experiment grid	pass
Submission hash matches metadata	pass
External files unchanged	pass

18. Repository

The complete project is published in the public GitHub repository. The SWEETTV_Midterm directory contains the project source, reports, verification artifacts and final submission output.

Project directory: https://github.com/chitransh-hse/SWEETTV-TV-Program-Recommendation/tree/main/SWEETTV_Midterm

Repository root: https://github.com/chitransh-hse/SWEETTV-TV-Program-Recommendation

Final submission and outputs: https://github.com/chitransh-hse/SWEETTV-TV-Program-Recommendation/tree/main/SWEETTV_Midterm/outputs

The original competition data are not included in the repository. The project documentation describes the expected data layout and execution steps. The final Kaggle submission file is outputs/final_submission.csv.

Appendix A. Complete validation grid

All 50 evaluated configurations, ordered by method and MAP@5. Runtime is the wall-clock time for that configuration only, excluding shared data loading.

Method	Configuration	MAP@5	Recall@5	Coverage	Runtime (s)
Popularity baseline	popularity users	0.572049	0.0470	0.08%	0.07
Popularity baseline	popularity sessions	0.537060	0.0454	0.08%	0.07
Popularity baseline	popularity events	0.499463	0.0429	0.08%	0.07
Content-based filtering	cbf genre log sessions k50	0.197621	0.0224	15.39%	0.46
Content-based filtering	cbf all log sessions k50	0.191245	0.0216	8.09%	0.61
Content-based filtering	cbf all log sessions k200	0.171872	0.0191	8.06%	0.66
Content-based filtering	cbf genre log sessions k200	0.169863	0.0190	22.24%	0.55
Content-based filtering	cbf all log sessions kall	0.128249	0.0144	8.46%	0.66
Content-based filtering	cbf genre binary k50	0.112516	0.0130	19.11%	0.50
Content-based filtering	cbf all binary k50	0.098765	0.0117	8.39%	0.57
Content-based filtering	cbf genre log sessions kall	0.084077	0.0097	22.16%	0.53
Content-based filtering	cbf all binary k200	0.073464	0.0087	8.50%	0.70
Content-based filtering	cbf genre binary k200	0.071605	0.0085	28.65%	0.59
Content-based filtering	cbf all binary kall	0.048147	0.0057	9.02%	0.65
Content-based filtering	cbf genre binary kall	0.038708	0.0049	24.17%	0.57
Association rules	ar confidence k50 sup5 pair5	0.570997	0.0469	0.16%	0.35
Association rules	ar confidence k100 sup3 pair3	0.570956	0.0469	0.17%	0.39
Association rules	ar confidence k50 sup3 pair3	0.570935	0.0469	0.19%	0.44
Association rules	ar lift k50 sup5 pair5	0.032721	0.0041	17.90%	0.40
Association rules	ar lift k50 sup3 pair3	0.025694	0.0033	16.67%	0.43

Method	Configuration	MAP@5	Recall@5	Coverage	Runtime (s)
Association rules	ar lift k100 sup3 pair3	0.023537	0.0031	16.86%	0.43
Repeat-recency personalisation	repeat tau120 alpha1.0	0.622277	0.0505	14.21%	0.18
Repeat-recency personalisation	repeat tau60 alpha1.5	0.622016	0.0504	14.30%	0.18
Repeat-recency personalisation	repeat tau90 alpha1.0	0.622016	0.0504	14.30%	0.18
Repeat-recency personalisation	repeat tau90 alpha1.5	0.621791	0.0505	14.13%	0.18
Repeat-recency personalisation	repeat tau120 alpha1.5	0.619726	0.0503	14.08%	0.18
Repeat-recency personalisation	repeat tau60 alpha1.0	0.618937	0.0502	14.44%	0.19
Hybrid (RRF)	hybrid 1.00 0.20 0.05 repeat recency + popularity + content based	0.634319	0.0514	10.25%	0.12
Hybrid (RRF)	hybrid 1.00 0.20 0.05 0.05 repeat recency + popularity + association rules + content based	0.632789	0.0513	9.41%	0.12
Hybrid (RRF)	hybrid rep tau60 a1.5 pop0.2	0.632035	0.0513	10.39%	0.10
Hybrid (RRF)	hybrid rep tau90 a1.0 pop0.2	0.632035	0.0513	10.39%	0.10
Hybrid (RRF)	hybrid rep tau120 a1.0 pop0.1	0.632004	0.0513	11.96%	0.10
Hybrid (RRF)	hybrid 1.00 0.20 0.10 repeat recency + popularity + content based	0.631765	0.0513	10.25%	0.11
Hybrid (RRF)	hybrid rep tau120 a1.0 pop0.2	0.631763	0.0513	10.28%	0.10
Hybrid (RRF)	hybrid rep tau60 a1.5 pop0.1	0.631753	0.0513	12.09%	0.10
Hybrid (RRF)	hybrid rep tau90 a1.0 pop0.1	0.631753	0.0513	12.09%	0.10
Hybrid (RRF)	hybrid rep tau90 a1.5 pop0.2	0.631671	0.0513	10.28%	0.10
Hybrid (RRF)	hybrid 1.00 0.20 0.10 repeat recency + popularity + association rules	0.631343	0.0512	8.97%	0.11
Hybrid (RRF)	hybrid rep tau90 a1.5 pop0.1	0.631280	0.0512	11.93%	0.10
Hybrid (RRF)	hybrid 1.00 0.20 0.05 repeat recency + popularity + association rules	0.631011	0.0512	9.51%	0.11
Hybrid (RRF)	hybrid rep tau60 a1.0 pop0.2	0.630739	0.0512	10.75%	0.10

Method	Configuration	MAP@5	Recall@5	Coverage	Runtime (s)
Hybrid (RRF)	hybrid rep tau60 a1.0 pop0.1	0.630189	0.0511	12.16%	0.10
Hybrid (RRF)	hybrid rep tau120 a1.5 pop0.2	0.630186	0.0512	10.31%	0.10
Hybrid (RRF)	hybrid rep tau120 a1.0 pop0.3	0.629907	0.0511	9.16%	0.10
Hybrid (RRF)	hybrid rep tau60 a1.0 pop0.3	0.629671	0.0511	9.46%	0.10
Hybrid (RRF)	hybrid rep tau60 a1.5 pop0.3	0.629542	0.0511	9.24%	0.10
Hybrid (RRF)	hybrid rep tau90 a1.0 pop0.3	0.629542	0.0511	9.24%	0.10
Hybrid (RRF)	hybrid rep tau90 a1.5 pop0.3	0.629517	0.0511	9.13%	0.10
Hybrid (RRF)	hybrid rep tau120 a1.5 pop0.1	0.629257	0.0511	11.96%	0.10
Hybrid (RRF)	hybrid rep tau120 a1.5 pop0.3	0.628352	0.0510	9.13%	0.10

Appendix B. Audit detail

The Stage 3 audit is an independent script that re-reads the artifacts rather than trusting the experiment outputs. It checks 33 conditions, all of which pass.

#	Check	Result
1	Stage 2.5 verification still 323/323	pass
2	source event store digest unchanged	pass
3	temporal split DB digest unchanged	pass
4	metadata projection digest unchanged	pass
5	main training data hash matches experiment	pass
6	main validation data hash matches experiment	pass
7	no test-period viewing log is used	pass
8	validation used for scoring/selection only	pass
9	selected model exists in experiment grid	pass
10	selected MAP@5 matches experiment	pass
11	experiment includes a popularity baseline	pass
12	experiment includes content-based filtering	pass
13	experiment includes association rules	pass
14	experiment includes a hybrid	pass
15	all reported metrics are finite	pass
16	submission header exact	pass
17	submission has exactly 1260 rows	pass
18	submission user IDs unique	pass
19	submission preserves template order	pass
20	every row has exactly five IDs	pass
21	no duplicate ID within a row	pass
22	no tv_show_id=0	pass

#	Check	Result
23	all IDs numeric	pass
24	all IDs in weeks 31-42 labelled universe	pass
25	test candidate universe size 6636	pass
26	submission hash matches metadata	pass
27	submission row count matches metadata	pass
28	selected model recorded in submission metadata	pass
29	external file unchanged: solution.csv	pass
30	external file unchanged: solution_improved.csv	pass
31	external file unchanged: final_submission.csv	pass
32	example recommendations cover two users	pass
33	example recommendations have five rows per user	pass